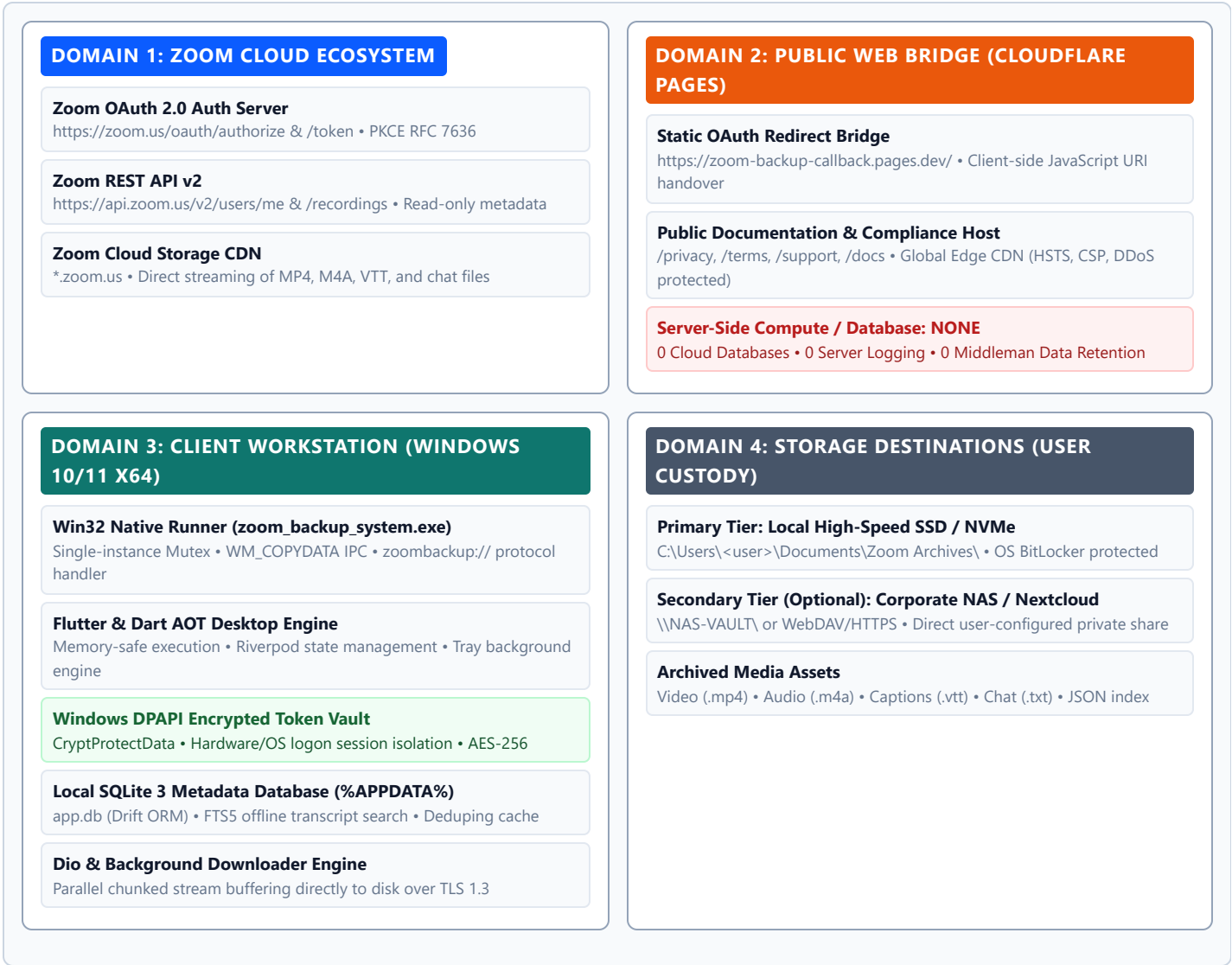


System Architecture & Data Flow Specification

Architectural Paradigm: Local-First Decentralized Archiving

Zoom BackUp operates under a **local-first, zero-knowledge architecture**. The developers host **zero multi-tenant backend servers, zero customer databases, and zero cloud media storage**. User meeting recordings, transcripts, and credentials move strictly between Zoom's secure cloud infrastructure and the user's authenticated workstation over TLS 1.3 encrypted sockets.

1. END-TO-END SYSTEM ARCHITECTURE DIAGRAM



Service / Component	Hosting / Provider	Interacts with Zoom?	Data Exchanged	Security & Cryptographic Controls
Zoom Media CDN	Zoom CDN (*.zoom.us)	YES (Direct)	Binary media streams (MP4, M4A, VTT, Chat)	TLS 1.3, temporary authenticated download tokens.
OAuth Callback Edge	Cloudflare Pages Global CDN	NO (Passive Bridge)	OAuth auth code in URL query string (redirect only)	Zero server-side database/logging; client-side URI handover to zoombackup://.
Local DPAPI Vault	Microsoft Windows OS Kernel	NO (Local Only)	Encrypted OAuth tokens & secrets	Windows CryptProtectData (AES-256), OS logon context bound.
Local SQLite Engine	Local Workstation (%APPDATA%)	NO (Local Only)	Meeting metadata index for offline search	Single-tenant local file system, zero remote access.
Corporate NAS / WebDAV	User Private Infrastructure (Optional)	NO (Post-Download)	Locally downloaded video/audio backup files	SMB/CIFS internal LAN or WebDAV over TLS; user-managed credentials.
Third-Party SaaS / Analytics	NONE (0 Services)	NO	Zero Data Exchanged	ZERO TELEMETRY — No trackers, analytics, or external DBs.

3. DATA FLOW DIAGRAM 1: OAUTH 2.0 PKCE AUTHENTICATION & HANDOVER

1 Authentication Initialization

User clicks "Sign In with Zoom" in Zoom BackUpz. Client generates a cryptographically random `code_verifier` and computes a SHA-256 `code_challenge` (RFC 7636 PKCE).

2 System Browser Launch

Client invokes system browser (via `url_launcher`) to `https://zoom.us/oauth/authorize` with parameters: `client_id`, `code_challenge`, `scope=recording:read user:read`, and `redirect_uri=https://zoom-backup-callback.pages.dev/`.

3 Zoom User Consent & Authorization

User logs into Zoom and reviews the least-privilege read-only permission screen. Upon approval, Zoom issues an authorization code and redirects the browser to the redirect URI.

4 Client-Side Deep Link Handover

The static page at `zoom-backup-callback.pages.dev` extracts code and executes a client-side URI handover to `zoombackup://oauth-callback?code=...`. Zero data is logged on the edge.

5 Win32 Protocol Activation & IPC Relaying

The Windows OS invokes the secondary executable instance registered to `zoombackup://`. The runner transmits the authorization payload to the running primary window using `WM_COPYDATA` with memory bounds verification.

6 Direct Token Exchange over TLS 1.3

The primary desktop application sends a direct POST request to `https://zoom.us/oauth/token` containing the authorization code and `code_verifier`. Zoom validates the PKCE challenge and returns access and refresh tokens.

7 Windows DPAPI Encrypted Vaulting

The received tokens are immediately encrypted at rest using the **Windows Data Protection API** (`CryptProtectData`) and stored in the user's OS vault. Plaintext tokens are purged from runtime memory.

4. DATA FLOW DIAGRAM 2: CLOUD RECORDING RETRIEVAL & DIRECT-TO-DISK ARCHIVING

1 Recording Metadata Query

Client decrypts access token in memory via DPAPI and executes GET `https://api.zoom.us/v2/users/me/recordings` over TLS 1.3. Zoom returns meeting list, timestamps, durations, and file stream inventories.

2 Local Deduping & Queue Scheduling

Client queries local SQLite database (`app.db`). If meeting files are already archived, they are skipped. New recordings are placed in the concurrent download queue.

3 Direct CDN Stream Buffering (Zero Cloud Intermediary)

Client connects directly to Zoom CDN (`*.zoom.us`) over TLS 1.3 using download tokens. High-speed streams (MP4 video, M4A audio, VTT captions, Chat logs) write directly to chunked buffers on the user's local disk.

4 Cryptographic Integrity Verification

Upon download completion, the client computes the SHA-256 checksum of each local file and compares it against Zoom's file manifest to guarantee uncorrupted data at rest.

5 Offline Full-Text Indexing

The VTT transcript captions and meeting metadata are parsed and indexed into SQLite FTS5, enabling millisecond-speed offline full-text search with zero internet access required.

5. DATA FLOW DIAGRAM 3: SESSION DEAUTHORIZATION & TOKEN PURGE

1 Deauthorization Trigger

Initiated when the user clicks "Sign Out & Purge Tokens" in Zoom BackUpz, or when access is revoked via the Zoom App Marketplace portal.

2 Instant DPAPI Credential Destruction (<1 millisecond)

The client immediately invokes secure deletion routines, overwriting and purging all DPAPI encrypted keys, access tokens, refresh tokens, and active session states from the workstation.

3 Token Revocation Notification

Client sends a final POST `https://zoom.us/oauth/revoke` to invalidate the token on Zoom's authentication servers.

4 User Data Sovereignty Preserved

Because zero customer data was ever stored on remote developer servers, 100% compliance with Zoom's deauthorization data erasure mandate is achieved instantly. The user retains all downloaded local files in their private storage folder.

6. TECHNICAL COMPLIANCE ATTESTATION

This architecture and data flow document accurately represents the operational mechanics of Zoom BackUpz v1.0.0. All interactions with Zoom API and CDN endpoints strictly enforce TLS 1.3 encryption and read-only least-privilege scopes.

J. Tyrell

Lead Systems & Security Architect
Zoom BackUpz Development Team

Security & Architecture Review Board

Attestation Date: September 18, 2026
Status: Verified Production Architecture